

Unidad Didáctica

Teoría de Grafos y Redes, Unidad IV

Contenido

Índice de Contenido	2
Introducción	3
Desarrollo de Contenidos	4
4.1. Conceptos básicos	4
4.2. Problema del Agente Viajero (TSP)	5
4.2.1 Heurístico del Vecino Más Cercano	8
4.2.2 Heurístico GRASP.	14
4.2.3 Metaheurístico Tabu Search	14
4.3. Algoritmos de ruta más corta sobre un grafo	23
4.3.1 Algoritmo de Disjktra	24
4.3.2 Algoritmo de Floy-Warshall	29
4.4. Algoritmo para flujo máximo en una red	37
4.4.1 Algoritmo de Ford y Fulkerson	37
4.5. Modelo de redes para proyectos	37
4.5.1 Técnica de Revisión y Evaluación de Proyectos (PERT)	37
4.5.2 Método de Ruta Crítica (CPM)	37
4.6. Aplicación de "Network Modelin" y "PERT_CPM" del WinOSB para la resolución de problemas de grafos y redes para proyectos contextualizados en la ingeniería de sistemas de información	37
Bibliografía y Webgrafía	37
Glosario	38

Introducción

La Teoría de Grafos y Redes constituye un área fundamental de la investigación operativa y la ciencia computacional, dedicada al estudio de estructuras que representan relaciones y conexiones entre elementos. Su versatilidad la convierte en una herramienta indispensable en la ingeniería en sistemas de información, ya que permite modelar y optimizar desde rutas de transporte y distribución, hasta redes de comunicación, flujos de datos y planificación de proyectos.

En esta unidad se abordarán los conceptos básicos de grafos y sus propiedades, así como la aplicación de algoritmos clásicos y modernos para resolver problemas de optimización en redes. Entre ellos, se estudiará el Problema del Agente Viajero (TSP) y sus soluciones heurísticas y metaheurísticas, los algoritmos de ruta más corta (Dijkstra, Floyd-Warshall), de flujo máximo (Ford-Fulkerson) y de árbol de expansión mínima (Kruskal, Prim). También se explorarán técnicas para la gestión de proyectos como PERT y CPM, esenciales para la programación y control de actividades interdependientes.

El aprendizaje se complementará con el uso de herramientas como Network Modeling y PERT_CPM de WinQSB y POM, que facilitan la modelación y resolución de problemas reales en entornos profesionales. Al finalizar, el estudiante será capaz de representar, analizar y optimizar sistemas complejos mediante modelos de grafos, aplicando soluciones eficientes y fundamentadas para mejorar la toma de decisiones en proyectos y operaciones propias de la ingeniería en sistemas de información.

Conceptos básicos

Un **grafo** es una estructura matemática que modela un conjunto de objetos, llamados **nod**os o **vértices**, y las conexiones entre ellos, llamadas **aristas** o **arcos**. Cuando las conexiones tienen dirección (como calles de un solo sentido), se llama **grafo dirigido**, y cuando no la tienen, es un **grafo no dirigido**.

Una **red** es esencialmente un grafo al que se le asignan valores o capacidades a sus conexiones, como distancias, costos, tiempos o flujos máximos. Esto permite modelar y resolver problemas de optimización del mundo real.

Para los problemas de ruta, es clave el concepto de **camino** o **ruta**, que es una secuencia de aristas que conectan un nodo de origen con uno de destino. El objetivo a menudo es encontrar el camino que **minimiza la suma de los pesos** de sus aristas (costo, distancia, tiempo). Cuando un camino visita todos los nodos de una red exactamente una vez y regresa al inicio, se llama **ciclo hamiltoniano**, que es la base del Problema del Agente Viajero (TSP).

En el contexto de la gestión de proyectos (PERT/CPM), el grafo se convierte en una **red de actividades**. Los nodos representan **eventos** (puntos de inicio o fin de tareas) y las aristas representan las **actividades** mismas, con un peso que indica su duración. La **ruta crítica** es entonces el camino más largo a través de esta red, determinando la duración mínima del proyecto.

Para resolver estos problemas, se utilizan dos tipos principales de algoritmos: los **algoritmos exactos**, que encuentran la solución óptima pero pueden ser lentos para problemas grandes, y los **algoritmos heurísticos**, que encuentran "buenas"

soluciones de forma mucho más rápida, aunque sin garantizar la optimalidad. Las metaheurísticas, como Búsqueda Tabú, son estrategias avanzadas que guían a los heurísticos para escapar de soluciones locales y explorar mejor el espacio de búsqueda.

Problema del Agente Viajero (TSP)

El **Problema del Agente Viajero (TSP)**, por sus siglas en inglés *Traveling Salesman Problem*) es uno de los desafíos más famosos y estudiados en el campo de la optimización combinatoria y la investigación de operaciones. Su objetivo fundamental es encontrar la **ruta más corta o de menor costo** para que un agente (por ejemplo, un vendedor, un repartidor o un vehículo) visite un conjunto de **puntos o ciudades** exactamente una vez y regrese al punto de partida.

Características Principales del TSP:

1. Estructura Básica:

- Se tiene un conjunto de **nodos** (ciudades, ubicaciones, puntos de entrega, etc.).
- Cada par de nodos está conectado por una **arista** que tiene asociado un **costo** (distancia, tiempo, gasto de combustible, etc.).
- El agente debe partir de un nodo inicial, visitar **todos los demás nodos una sola vez** y finalmente regresar al nodo de origen.

2. Complejidad Computacional:

- El TSP pertenece a la clase de problemas **NP-duros**, lo que significa que, a medida que el número de nodos aumenta, el tiempo requerido para encontrar la solución óptima crece de manera exponencial.
- Para n ciudades, existen $(n-1)!/2$ rutas posibles en el caso simétrico (cuando el costo de ir de A a B es el mismo que de B a A). Por ejemplo,

con solo 10 ciudades, hay aproximadamente 181,440 rutas posibles; para 20 ciudades, este número supera los 10181018.

3. Representación Matemática:

- El problema puede modelarse mediante **grafos**, donde los nodos representan las ciudades y las aristas los caminos entre ellas con sus respectivos costos.
- La solución óptima es un **ciclo hamiltoniano** de costo mínimo en el grafo, es decir, un recorrido que pasa por todos los vértices una sola vez y regresa al inicio.

4. Variantes y Contextos de Aplicación:

- **Logística y Distribución:** Optimización de rutas de reparto para minimizar distancias recorridas, tiempo de entrega o consumo de combustible.
- **Manufactura:** Secuenciación de tareas en máquinas para reducir tiempos de cambio entre órdenes de producción.
- **Electrónica:** Diseño de circuitos impresos para minimizar la longitud de las trayectorias de perforación.
- **Genética:** Construcción de mapas genómicos mediante la integración de segmentos de ADN.
- **Astronomía:** Planificación de trayectorias para satélites o telescopios espaciales que deben observar múltiples objetivos celestes.

5. Desafíos Inherentes:

- La **explosión combinatoria** hace inviable la enumeración exhaustiva de todas las rutas para instancias grandes.
- Factores del mundo real agregan capas de complejidad, como:
 - Restricciones de tiempo (ventanas de entrega).
 - Capacidades limitadas de vehículos.
 - Condiciones dinámicas (tráfico, clima).

- Múltiples objetivos simultáneos (costo, tiempo, sostenibilidad).

6. Impacto Práctico:

- Pese a su complejidad teórica, el TSP tiene aplicaciones críticas en industrias como el transporte, la cadena de suministro y la planificación de proyectos. Resolverlo eficientemente permite a las empresas reducir costos operativos, mejorar la satisfacción del cliente y minimizar su impacto ambiental.

En resumen, el TSP es un problema paradigmático que encapsula la esencia de la optimización bajo restricciones, desafiando a investigadores y profesionales a encontrar equilibrios entre la calidad de la solución y los recursos computacionales requeridos. Su estudio continúa impulsando avances en algoritmos, inteligencia artificial y gestión logística.

Heurístico del Vecino Más Cercano

El **Heurístico del Vecino Más Cercano** es un algoritmo greedy (voraz) utilizado para resolver el Problema del Agente Viajero (TSP) de manera aproximada. Su objetivo es construir una ruta paso a paso, seleccionando en cada iteración la ciudad no visitada más cercana a la ciudad actual. Aunque no garantiza la solución óptima, es computacionalmente eficiente y fácil de implementar, lo que lo hace útil en aplicaciones prácticas donde se necesitan soluciones rápidas y razonables.

Funcionamiento Detallado del Heurístico

A continuación, se describe el proceso paso a paso, junto con un ejemplo ilustrativo y una aplicación profunda en logística.

Pasos del Algoritmo:

1. Inicialización:

- Seleccionar una ciudad de inicio (puede ser arbitraria o basada en criterios específicos).
- Marcar esta ciudad como "visitada".
- Inicializar la ruta con esta ciudad.

2. Iteración:

- Desde la ciudad actual, examinar todas las ciudades no visitadas y elegir la más cercana (es decir, la que tenga la menor distancia o costo).
- Mover a esa ciudad, marcarla como visitada y agregarla a la ruta.
- Repetir este paso hasta que todas las ciudades hayan sido visitadas.

3. Cierre del ciclo:

- Una vez visitadas todas las ciudades, regresar a la ciudad de inicio para completar el ciclo hamiltoniano.

4. Cálculo del costo total:

- Sumar las distancias o costos de todas las aristas recorridas en la ruta.

Ejemplo Básico con 4 Ciudades

Supongamos un TSP con ciudades **A**, **B**, **C**, **D** y la siguiente matriz de distancias (simétrica):

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

Aplicamos el heurístico empezando desde **A**:

- **Paso 1:** Inicio en **A**. Ciudades no visitadas: B, C, D. La más cercana es **B** (distancia 10). Ruta: $A \rightarrow B$.
- **Paso 2:** Desde **B**, ciudades no visitadas: C, D. La más cercana es **D** (distancia 25). Ruta: $A \rightarrow B \rightarrow D$.
- **Paso 3:** Desde **D**, ciudad no visitada: **C** (distancia 30). Ruta: $A \rightarrow B \rightarrow D \rightarrow C$.
- **Paso 4:** Regresar a **A** desde **C** (distancia 15). Ruta final: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$.
- **Costo total:** $10 + 25 + 30 + 15 = 80$.

Esta solución puede no ser óptima (por ejemplo, la ruta $A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$ tiene costo $15 + 30 + 25 + 10 = 80$, pero en otros casos la diferencia puede ser significativa).

Ejemplo: Aplicación en Logística de Última Milla

Imaginemos una empresa de reparto que debe entregar paquetes en **5 ubicaciones** en una ciudad. Las ubicaciones son: **Almacén (A)**, **Oficina (B)**, **Escuela (C)**, **Hospital (D)**, y **Centro Comercial (E)**. La matriz de distancias en km es:

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>A</i>	0	5	10	8	12
<i>B</i>	5	0	3	7	9
<i>C</i>	10	3	0	4	6
<i>D</i>	8	7	4	0	5
<i>E</i>	12	9	6	5	0

Aplicamos el heurístico desde el **Almacén (A)**:

- **Paso 1:** Inicio en **A**. Ciudades no visitadas: B, C, D, E. La más cercana es **B** (5 km). Ruta: A → B.
- **Paso 2:** Desde **B**, no visitadas: C, D, E. La más cercana es **C** (3 km). Ruta: A → B → C.
- **Paso 3:** Desde **C**, no visitadas: D, E. La más cercana es **D** (4 km). Ruta: A → B → C → D.
- **Paso 4:** Desde **D**, no visitada: **E** (5 km). Ruta: A → B → C → D → E.
- **Paso 5:** Regresar a **A** desde **E** (12 km). Ruta final: A → B → C → D → E → A.
- **Costo total:** $5 + 3 + 4 + 5 + 12 = 29$ km.

Esta ruta genera un plan de entregas eficiente en términos de distancia, pero no necesariamente es la óptima (por ejemplo, la ruta A → D → E → C → B → A tiene costo $8 + 5 + 6 + 3 + 5 = 27$ km, que es mejor). Sin embargo, el heurístico proporciona una solución rápida y usable.

Aplicación Profunda para Soluciones TSP en Contextos Reales

El heurístico del vecino más cercano se utiliza como **punto de partida** en estrategias más complejas para TSP, especialmente en logística, donde la escalabilidad y la velocidad son cruciales. A continuación, se describe cómo se integra en un framework de solución para un caso de **flota de reparto con 100 clientes**.

Escenario:

Una empresa de comercio electrónico debe planificar las rutas diarias para 10 vehículos que reparten paquetes a 100 clientes en una zona metropolitana. El objetivo es minimizar la distancia total recorrida, considerando restricciones como ventanas de tiempo de entrega y capacidad de los vehículos.

Proceso de Solución usando el Heurístico:

1. Preprocesamiento de Datos:

- Recopilar las direcciones de los clientes y el almacén central.
- Calcular una matriz de distancias realistas usando APIs de mapas (e.g., Google Maps), considerando tráfico y carreteras.
- Agrupar clientes por zonas geográficas para asignarlos a vehículos (esto simplifica el problema en múltiples TSP más pequeños).

2. Aplicación del Heurístico por Vehículo:

- Para cada vehículo, se selecciona un subconjunto de clientes (por ejemplo, 10 clientes por vehículo).
- Se aplica el heurístico del vecino más cercano de forma independiente para cada vehículo, iniciando desde el almacén.
- **Ejemplo para un vehículo:**
 - Inicio en Almacén.

- Se elige el cliente más cercano no visitado, se marca como visitado y se agrega a la ruta.
- Se repite hasta que todos los clientes asignados sean visitados.
- Se regresa al almacén.
- Esto genera rutas iniciales rápidamente.

3. Mejora de la Solución:

- El heurístico alone puede producir rutas subóptimas debido a su naturaleza greedy. Por lo tanto, se combina con técnicas de optimización:
 - **Búsqueda Local:** Aplicar algoritmos como **2-opt** o **3-opt** para refinar las rutas intercambiando aristas y reduciendo cruces.
 - **Metaheurísticas:** Usar la ruta del vecino más cercano como solución inicial en algoritmos como **Búsqueda Tabú** o **Algoritmos Genéticos** para explorar el espacio de soluciones de manera más amplia.
 - **Optimización en Tiempo Real:** En sistemas dinámicos, el heurístico se re-ejecuta periódicamente para ajustar las rutas ante nuevos pedidos o cambios en el tráfico.

4. Evaluación y Implementación:

- Calcular el costo total (distancia, tiempo, combustible) de las rutas generadas.
- Simular las rutas en un entorno de prueba para validar su factibilidad.
- Desplegar las rutas en dispositivos móviles de los conductores, con actualizaciones en tiempo real.

Ventajas y Limitaciones en este Contexto:

- **Ventajas:**

- **Rapidez:** El heurístico es $O(n^2)$, manejable incluso para 100 clientes por vehículo.
- **Simplicidad:** Fácil de implementar y entender, lo que facilita su integración en sistemas de planificación.
- **Flexibilidad:** Se adapta a restricciones adicionales (e.g., priorizar clientes cercanos en tiempo real).

- **Limitaciones:**

- **Suboptimalidad:** Puede omitir rutas globalmente óptimas debido a decisiones locales.
- **Sensibilidad al punto de inicio:** La solución varía según la ciudad inicial; una práctica común es ejecutarlo múltiples veces desde diferentes puntos y elegir la mejor ruta.

El heurístico del vecino más cercano es una herramienta fundamental en el arsenal de métodos para TSP, especialmente en aplicaciones logísticas donde la velocidad es crítica. Aunque no garantiza optimalidad, su eficiencia lo convierte en un componente valioso para generar soluciones iniciales que luego son refinadas por técnicas más avanzadas. En problemas del mundo real, como la planificación de rutas de reparto, su uso práctico se maximiza cuando se combina con estrategias de optimización y adaptación dinámica, permitiendo a las empresas reducir costos y mejorar la eficiencia operativa.

Heurístico GRASP.

GRASP (Greedy Randomized Adaptive Search Procedure, o en español, Procedimiento Codicioso de Búsqueda Adaptativa y Aleatorizada) es un metaheurístico utilizado para resolver problemas de optimización combinatoria, especialmente aquellos que son NP-difíciles, como el TSP.

La idea principal de GRASP es combinar:

- Una fase constructiva (que genera soluciones iniciales de buena calidad de manera aleatorizada).
- Una fase de búsqueda local (que mejora iterativamente esas soluciones).

Fases de GRASP:

a) Fase Constructiva:

- Se construye una solución paso a paso.
- En cada paso, se crea una Lista Restringida de Candidatos (RCL) con los mejores elementos según una función greedy, pero no necesariamente el mejor absoluto.
- Se elige aleatoriamente un elemento de la RCL.
- Esto introduce diversidad, evitando que siempre se genere la misma solución.

b) Fase de Búsqueda Local:

- Sobre la solución construida, se aplica un método de mejora (ej: 2-opt para TSP) para optimizarla localmente.
- Si se encuentra una solución mejor, se actualiza la mejor solución global.

c) Iteraciones:

- Se repiten las fases constructivas y de búsqueda local múltiples veces.
- Al final, se devuelve la mejor solución encontrada en todas las iteraciones.

Aplicación de GRASP al TSP

El TSP consiste en encontrar el recorrido más corto que visite cada ciudad exactamente una vez y regrese al origen.

Pasos para aplicar GRASP al TSP:

a) Fase Constructiva:

1. Inicialización: Empezar desde una ciudad aleatoria.
2. Construcción de la RCL:
 - Desde la ciudad actual, consideramos las ciudades no visitadas.
 - Calculamos las distancias a ellas.
 - Ordenamos las ciudades por distancia (menor a mayor).
 - Creamos la RCL con las k ciudades más cercanas (o usando un umbral α).
3. Selección aleatoria: Elegimos una ciudad al azar de la RCL y la añadimos al recorrido.
4. Repetir hasta visitar todas las ciudades.

b) Fase de Búsqueda Local:

- Aplicamos 2-opt (intercambiar dos aristas para reducir la distancia) u otra heurística de mejora.
- Iteramos hasta que no haya mejora.

c) Iteraciones:

- Repetimos el proceso (fase constructiva + búsqueda local) N veces.
- Nos quedamos con el mejor recorrido encontrado.

Ejemplo Simplificado:

Supongamos 5 ciudades con distancias conocidas.

1. Fase Constructiva:

- Empezamos en ciudad A.
- RCL con las 2 ciudades más cercanas a A (ej: B, C).
- Elegimos al azar B.
- Desde B, RCL con ciudades no visitadas (ej: D, E), elegimos D.
- Continuamos hasta completar el recorrido: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow A$.

2. Fase de Búsqueda Local (2-opt):

- Probamos intercambios de aristas: Ej: Cambiar (B-D) y (E-C) por (B-E) y (D-C). Si mejora, aceptamos.
- Repetimos hasta no poder mejorar.

3. Iteraciones:

- Repetimos desde la fase constructiva (quizás empezando desde otra ciudad) varias veces.
- Nos quedamos con el mejor recorrido global.

Ventajas de GRASP para TSP:

- No deterministicidad: Al tener aleatoriedad, explora distintas zonas del espacio de soluciones.
- Flexibilidad: Se adapta fácilmente a variantes del TSP.
- Calidad de solución: Suele dar mejores resultados que métodos puramente greedy.

Desventajas:

- Tiempo de ejecución: Múltiples iteraciones pueden ser costosas.
- Parámetros: Elección de α (tamaño de RCL) y número de iteraciones afecta el resultado.

Metaheurístico Tabu Search

La Búsqueda Tabú es una metaheurística de mejora iterativa diseñada para guiar una búsqueda local más allá de los óptimos locales. Su idea central es evitar ciclos y explorar nuevas regiones de la solución prohibiendo (haciendo "tabú") los movimientos inversos a los realizados recientemente.

Imagina que estás en una montaña (el espacio de soluciones) y quieres encontrar el pico más alto (la solución óptima). Un algoritmo de ascenso de colina simple (hill-climbing) se movería siempre hacia la cima más cercana. Pero si esa cima es baja (óptimo local), se atasca. La Búsqueda Tabú te dice: "Está bien, baja un poco de esta colina, y para asegurarte de no volver a subir por el mismo camino inmediatamente, prohíbe dar los pasos que acabas de dar". Esto te fuerza a buscar un camino alternativo que quizás te lleve a una montaña mucho más alta.

Componentes Clave de la Búsqueda Tabú:

1. Solución Inicial: Puede ser una solución aleatoria o, como en tu caso, una buena solución generada por el Vecino Más Cercano o GRASP.
2. Vecindario: El conjunto de soluciones "ceranas" a la solución actual que se pueden generar mediante un movimiento simple (e.g., intercambiar dos ciudades en el TSP).
3. Lista Tabú: Es el corazón del algoritmo. Es una memoria a corto plazo que almacena los movimientos recientes (o sus atributos) que están prohibidos. Su objetivo es evitar volver a una solución ya visitada y prevenir ciclos.
 - Ejemplo: Si intercambiaste las ciudades A y B, la lista tabú podría prohibir volver a intercambiarlas por un número determinado de iteraciones.
4. Criterio de Aspiración: Una regla que anula la condición tabú. Si un movimiento tabú lleva a una solución que es mejor que la mejor solución global encontrada hasta el momento, se acepta de todos modos. "Vale la pena romper la regla por un resultado excepcional".

5. Función Objetivo: La métrica que se busca optimizar (en TSP, la distancia total del recorrido).

6. Criterio de Parada: Puede ser un número máximo de iteraciones, un número de iteraciones sin mejora, o que se alcance un valor objetivo.

Solución al TSP usando Búsqueda Tabú

Ahora, aplicamos estos conceptos directamente al Problema del Vendedor Viajero (TSP).

Objetivo: Encontrar el recorrido (permutación de ciudades) que minimice la distancia total.

Paso a Paso del Algoritmo:

1. Inicialización:

- Genera una solución inicial, S_{actual} . Gracias a tus conocimientos previos, puedes usar:
 - S_{actual} = Algoritmo del Vecino Más Cercano (rápido y decente).
 - S_{actual} = Solución de GRASP (mejor calidad, más diversa).
- Define $S_{mejor} = S_{actual}$ y $f(S_{mejor}) = f(S_{actual})$ (donde f es la distancia total).
- Inicializa la Lista Tabú (LT) como vacía.

2. Definir la Estructura de Vecindario y Movimientos:

- Un movimiento común en TSP es el intercambio 2-opt o swap de dos ciudades. Es decir, seleccionar dos ciudades en el recorrido e intercambiar sus posiciones.

- El vecindario de s_{actual} es el conjunto de todas las soluciones que se pueden generar aplicando un solo movimiento de intercambio de dos ciudades.

3. Configurar la Lista Tabú:

- Necesitamos decidir QUÉ se almacena en la lista tabú. No es eficiente almacenar soluciones completas (son muy grandes para comparar). En su lugar, almacenamos atributos del movimiento.
- Para un movimiento de intercambio entre las ciudades i y j , lo más común es almacenar el par (i, j) en la lista tabú. Esto significa que, mientras esté en la lista, está prohibido volver a intercambiar esas mismas dos ciudades.
- Define una longitud de la lista tabú (tenure). Por ejemplo, 7, 15 o $\sqrt{n}$ (donde n es el número de ciudades). Esta es la cantidad de iteraciones que un movimiento permanece en la lista.

4. Bucle Principal (Iteraciones):

Se repite hasta cumplir el criterio de parada(e.g., 1000 iteraciones).

a. Generar Vecindario Candidato: Examina todos (o una muestra) de los movimientos posibles del vecindario de S_{actual} .

b. Evaluar y Seleccionar: Encuentra el mejor movimiento admisible en el vecindario.

Un movimiento es admisible si:

1. NO está en la Lista Tabú, O
2. SÍ está en la Lista Tabú, pero cumple el Criterio de Aspiración (la solución resultante es mejor que S_{mejor}).

c. Actualizar:

- Aplica el mejor movimiento admisible a s_{actual} para crear s_{nueva} .
- Actualiza $s_{actual} = s_{nueva}$.
- Si $f(s_{actual}) < f(s_{mejor})$, entonces actualiza $s_{mejor} = s_{actual}$.

- Añade el movimiento que acabas de hacer (el par de ciudades intercambiadas) a la Lista Tabú. Si la lista ya está llena, elimina el elemento más antiguo (FIFO: First-In, First-Out).

d. Avanzar: Continúa con la siguiente iteración.

Ejemplo Concreto y Visual (Mini-TSP)

Supongamos 4 ciudades: A, B, C, D.

Solución inicial: A-B-C-D-A (Distancia: 100). $S_{\text{mejor}} = \text{A-B-C-D-A}$.

Longitud Lista Tabú = 2.

Iteración S_{actual} Mejor Movimiento Admissible ¿Es Tabú? ¿Cumple Aspiración?

Nueva S_{actual} Nueva Distancia Lista Tabú (más nuevo...) S_{mejor}

- * 0 A-B-C-D-A - - - - 100 (vacía) A-B-C-D-A (100)
- * 1 A-B-C-D-A Intercambiar B y C No - A-C-B-D-A 95 (B, C) A-C-B-D-A (95)
- * 2 A-C-B-D-A Intercambiar C y D No - A-D-B-C-A 98 (C, D), (B, C) A-C-B-D-A (95)
- * 3 A-D-B-C-A Intercambiar B y C Sí No (98 > 95) - - (C, D), (B, C) A-C-B-D-A (95)
- * Siguiendo mejor: Inter. A y D No - D-A-B-C-D 110 (A, D), (C, D) A-C-B-D-A (95)
- * 4 D-A-B-C-D Intercambiar A y B No - D-B-A-C-D 93 (A, B), (A, D) D-B-A-C-D (93)

¿Qué pasó aquí?

- En la Iteración 3, el mejor movimiento era volver a intercambiar B y C (que nos devolvería a la solución de la iteración 1). Pero este movimiento estaba en la lista tabú y no cumplía el criterio de aspiración, así que fue rechazado.
- Esto forzó al algoritmo a elegir el segundo mejor movimiento (intercambiar A y D), que era peor ($110 > 95$). ¡Aceptó empeorar la solución! Esto le permitió "escapar" de la zona del óptimo local.
- Al moverse a una solución peor, encontró un nuevo camino que en la Iteración 4 llevó a una solución globalmente mejor (93). El criterio de aspiración se activó y se actualizó s_{mejor} .

Resumen de Ventajas para el TSP:

- Evita Óptimos Locales: Al permitir movimientos que empeoran la solución, explora más a fondo el espacio de búsqueda.
- Es Eficiente y Flexible: La lista tabú por atributos es simple de implementar y gestionar.
- Potencia Combinada: Funciona excepcionalmente bien cuando se inicializa con un buen algoritmo constructivo (como los que ya conoces), ya que parte de una base sólida y se dedica a refinarla intensamente.

La Búsqueda Tabú es un gran paso adelante desde GRASP, ya que añade una memoria explícita (la lista tabú) que guía la búsqueda de manera más inteligente, en lugar de depender solo de la aleatoriedad para escapar de los óptimos locales.

Algoritmos de ruta más corta sobre un grafo

Los algoritmos de ruta más corta son herramientas centrales en ingeniería de sistemas: se aplican en redes de comunicaciones, planificación de rutas, logística, GIS, robótica y más. En esta sección se estandariza la notación, se presentan dos algoritmos clásicos (Dijkstra y Floyd–Warshall) con pseudocódigo, complejidad, esbozos de corrección, ejemplos (incluyendo uno contextualizado en Nicaragua), soluciones comentadas a los ejercicios propuestos y recomendaciones prácticas para implementar y visualizar grafos.

Convenciones y notación

- Grafo: $G = (V, E)$ donde V es el conjunto de vértices (nodos) y $E \subseteq V \times V$ el conjunto de aristas (arcos).
- Tamaños: $n = |V|$, $m = |E|$.
- Peso de una arista: $w(u, v)$ (si la arista no existe, se interpreta como $w(u, v) = \infty$). Si el grafo es no dirigido, $w(u, v) = w(v, u)$.
- Camino (path): secuencia de vértices conectados por aristas.
- Distancia mínima: $d(u, v)$ es el menor costo (peso) de un camino de u a v ; si no existe camino, $d(u, v) = \infty$.
- Símbolos: se usará consistentemente ∞ para infinito y matrices con encabezados de filas y columnas alineadas.
- Grafos dirigidos/ no dirigidos: explícelo siempre. Si no se indica, se entenderá que el grafo es dirigido. Para grafos no dirigidos puede modelarse cada arista $\{u, v\}$ como dos arcos (u, v) y (v, u) con mismo peso.

Algoritmo de Dijkstra

Propósito y supuestos

- Propósito: Obtener $d(s, v)$ para todo $v \in V$ a partir de un origen s .
- Supuesto crítico: **todos los pesos** $w(u, v)$ **deben ser** ≥ 0 . Si hay pesos negativos, Dijkstra puede fallar.
- Aplicable a grafos dirigidos y no dirigidos; complejidad depende de la estructura de datos para el conjunto de candidatos.

Descripción informal

Dijkstra es un algoritmo voraz que, a partir del nodo origen, va fijando de forma definitiva la distancia mínima de algunos nodos (los más cercanos aún no definitivos) y relaja aristas para actualizar distancias tentativas.

Pseudocódigo (versión con heap/cola de prioridad)

Entrada: $G=(V,E)$, pesos $w(u,v) \geq 0$, origen s
Salida: $\text{dist}[v] = d(s,v)$ (∞ si no alcanzable), $\text{prev}[v]$ = predecesor en la ruta mínima

```
Dijkstra(G, w, s):  
  for v in V:  
    dist[v] :=  $\infty$   
    prev[v] := NIL  
  dist[s] := 0  
  
  Q := priority_queue() # min-heap por dist[]  
  for v in V:  
    Q.insert(v, dist[v])  
  
  while Q not empty:  
    u := Q.extract_min() # nodo con menor dist[u] provisional  
    for each (u,v) in E: # para cada vecino v de u  
      alt := dist[u] + w(u,v)  
      if alt < dist[v]:  
        dist[v] := alt  
        prev[v] := u  
        Q.decrease_key(v, alt)  
  
  return dist, prev
```


Complejidad temporal y espacial

- Con cola de prioridad binaria (heap): $O((n + m)\log n)$.
 - Insertar n vértices en el heap: $O(n)$ o $O(n\log n)$ según implementación.
 - m relajaciones $\rightarrow$ hasta m llamadas a `decrease_key`: $O(m\log n)$.
- Con heap de Fibonacci: $O(n\log n + m)$.
- Implementación simple con matriz (o cola no optimizada): $O(n^2)$.
- Espacio: $O(n + m)$ (almacenar grafo + arrays `dist` y `prev`).

Explicación línea por línea

- Inicializamos `dist` a ∞ excepto el origen. `prev` guarda la predecesora para reconstruir rutas.
- Inserción en el heap permite extraer el nodo con menor distancia provisional rápidamente.
- En cada extracción, la distancia de u ya es definitiva (invariante del algoritmo). Relajamos aristas (u, v) : si al pasar por u mejoramos v , actualizamos `dist[v]` y su posición en la cola.
- Repetir hasta que no queden nodos por procesar.

Esbozo de corrección (proof sketch)

- **Invariante:** cuando se extrae u de la cola (es decir, se hace `extract_min`), `dist[u]` es la distancia mínima definitiva $d(s, u)$.
 - Razón: por contradicción, si existe un camino más corto a u pasando por algún vértice no retirado aún, entonces habría un vértice en ese camino con distancia provisional menor, lo que contradice que u fuera el mínimo actualmente.
- Relajaciones garantizan que las mejores distancias se propagan desde el origen hacia los demás nodos.

Reconstrucción de la ruta

Pseudocódigo:

```
ReconstructPath(prev, t):  
    path := []  
    u := t  
    while u != NIL:  
        path.prepend(u)  
        u := prev[u]  
    if path[0] != s:  
        return "No hay camino"  
    return path
```

prev surgido de Dijkstra permite reconstruir de destino hacia origen.

Ejemplo mejorado: el grafo original A→F (paso a paso con tabla dist/prev)

Grafo (no dirigido, pesos no negativos)

$G = (V, E)$, $V = \{A, B, C, D, E, F\}$

E y pesos $w(u, v)$:

$(A, B)=8$, $(A, D)=2$,

$(B, C)=1$, $(B, E)=3$,

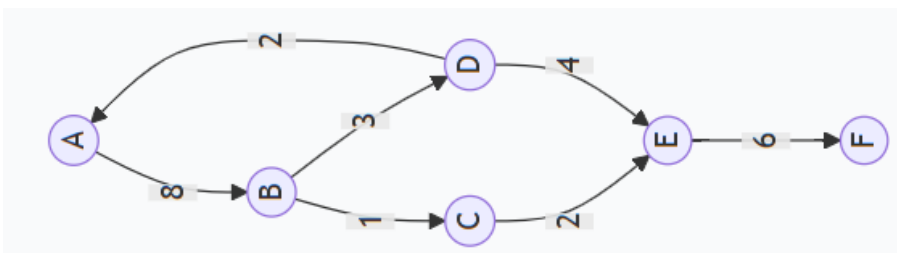
$(C, F)=2$,

$(D, E)=4$,

$(E, F)=6$

Código Mermaid para mostrar en Markdown

```
graph TD  
    A((A)) --8--> B((B))  
    B((B)) --1--> C((C))  
    B((B)) --3--> D((D))  
    C((C)) --2--> E((E))  
    D((D)) --4--> E((E))  
    D((D)) --2--> A((A))  
    E((E)) --6--> F((F))
```



(Ver arriba: A-B 8, A-D 2, B-C 1, B-E 3, C-F 2, D-E 4, E-F 6)

Ejecución de Dijkstra (origen A). Tabla por iteración

Inicial:

- $\text{dist}[A]=0, \text{dist}[B]=\infty, \text{dist}[C]=\infty, \text{dist}[D]=\infty, \text{dist}[E]=\infty, \text{dist}[F]=\infty$
- $\text{prev}[*]=\text{NIL}$
- Conjunto $Q=\{A,B,C,D,E,F\}$

Iteración 1: extraer A (dist 0)

- Relajar (A,B): $\text{alt} = 0 + 8 = 8 \rightarrow \text{dist}[B]=8, \text{prev}[B]=A$
- Relajar (A,D): $\text{alt} = 0 + 2 = 2 \rightarrow \text{dist}[D]=2, \text{prev}[D]=A$

Estado después:

- $\text{dist}: A:0, B:8, C:\infty, D:2, E:\infty, F:\infty$
 - $\text{prev}: A:\text{NIL}, B:A, C:\text{NIL}, D:A, E:\text{NIL}, F:\text{NIL}$
- $Q=\{B,C,D,E,F\}$

Iteración 2: extraer D (dist 2)

- Relajar (D,E): $\text{alt} = 2 + 4 = 6 \rightarrow \text{dist}[E]=6, \text{prev}[E]=D$

Estado:

- $\text{dist}: A:0, B:8, C:\infty, D:2, E:6, F:\infty$
 - $\text{prev}: B:A, D:A, E:D$
- $Q=\{B,C,E,F\}$

Iteración 3: extraer E (dist 6) — **atención:** en su ejemplo original se extrajo B antes, pero E tiene $6 < 8$, por lo tanto el orden correcto es extraer E antes que B. (Aquí corregimos la secuencia original.)

- Relajar (E,F): $\text{alt} = 6 + 6 = 12 \rightarrow \text{dist}[F]=12, \text{prev}[F]=E$
- Relajar (E,B): $\text{alt} = 6 + 3 = 9 \rightarrow \text{dist}[B]$ actual 8 $\rightarrow$ no se actualiza

Estado:

- $\text{dist}: A:0, B:8, C:\infty, D:2, E:6, F:12$
 - $\text{prev}: B:A, C:\text{NIL}, F:E$
- $Q=\{B,C,F\}$

Iteración 4: extraer B (dist 8)

- Relajar (B,C): $\text{alt} = 8 + 1 = 9 \rightarrow \text{dist}[C]=9, \text{prev}[C]=B$
- Relajar (B,E): $\text{alt} = 8 + 3 = 11 \rightarrow \text{dist}[E]=6 \rightarrow \text{no cambia}$

Estado:

- dist: A:0, B:8, C:9, D:2, E:6, F:12
- prev: C:B
- $Q=\{C,F\}$

Iteración 5: extraer C (dist 9)

- Relajar (C,F): $\text{alt} = 9 + 2 = 11 \rightarrow \text{dist}[F] = \min(12,11) = 11 \rightarrow \text{actualiza}$
 $\text{prev}[F]=C$

Estado:

- dist: A:0, B:8, C:9, D:2, E:6, F:11
- prev: F:C
- $Q=\{F\}$

Iteración 6: extraer F (dist 11) — sin vecinos nuevos que mejoren distancias.

Resultado final:

- Distancia mínima $d(A,F) = 11$.
- prev:

Nodo prev

A	NIL
B	A
C	B
D	A
E	D
F	C

Reconstrucción (A→F): $F \leftarrow C \leftarrow B \leftarrow A \Rightarrow \text{ruta } A \rightarrow B \rightarrow C \rightarrow F$.

Observación: la corrección del orden de extracción (E antes que B) muestra la importancia de elegir siempre el nodo con menor dist en Q. Buen ojo si un alumno detecta discrepancias en una ejecución manual: muchas veces el error está en no ordenar correctamente las extracciones.

Algoritmo de Floyd-Warshall

Propósito y supuestos

- Propósito: calcular todas las distancias mínimas entre todos los pares de vértices $d(u, v)$ para $u, v \in V$.
- Soporta pesos negativos en las aristas, **pero no debe haber ciclos de peso negativo** (si existen, la matriz final indica la presencia de dichos ciclos por entradas diagonales negativas).
- Basado en programación dinámica: considera progresivamente vértices intermedios permitidos.

Pseudocódigo

Entrada: $n = |V|$, $w(u, v)$ para todos u, v (∞ si no hay arista), índice de vértices $1..n$

Salida: $D[][]$ con distancias mínimas entre todos los pares

```
FloydWarshall(w):
    for i in 1..n:
        for j in 1..n:
            if i == j:
                D[i][j] := 0
            else if (i, j) in E:
                D[i][j] := w(i, j)
            else:
                D[i][j] :=  $\infty$ 

    for k in 1..n:
        for i in 1..n:
            for j in 1..n:
                if D[i][k] + D[k][j] < D[i][j]:
                    D[i][j] := D[i][k] + D[k][j]

    return D
```

Complejidad

- Temporal: $O(n^3)$.
- Espacial: $O(n^2)$ para la matriz D .
- Es simple, robusto y útil para grafos densos o cuando se requiere información de todos los pares.

Esbozo de corrección (proof sketch)

- Definir $D^{(k)}[i][j]$ como la longitud mínima de un camino de i a j cuyos vértices intermedios (si los hay) pertenecen al conjunto $\{1, \dots, k\}$.
- Recurrencia: $D^{(k)}[i][j] = \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j])$.
- Inicialización $D^{(0)}$ es la matriz de adyacencia con 0 en la diagonal y ∞ si no hay arista.
- Al finalizar con $k = n$ se consideran todos los vértices como intermedios y por tanto $D^{(n)}$ contiene las distancias mínimas.

Detección de ciclos negativos

Si para algún i , $D[i][i] < 0$ al final, existe un ciclo de peso negativo alcanzable desde i .

Ejemplo con Floyd–Warshall (matriz inicial y resultado final)

Usando el grafo A,B,C,D,E,F previo. Presentamos la matriz inicial $D^{(0)}$ en tabla:

Matriz $D^{(0)}$

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
<i>A</i>	0	8	∞	2	∞	∞
<i>B</i>	8	0	1	∞	3	∞
<i>C</i>	∞	1	0	∞	∞	2
<i>D</i>	2	∞	∞	0	4	∞
<i>E</i>	∞	3	∞	4	0	6
<i>F</i>	∞	∞	2	∞	6	0

Después de aplicar Floyd–Warshall (se pueden mostrar iteraciones por k , pero por brevedad se muestra la matriz final):

Matriz final D

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
<i>A</i>	0	8	9	2	6	11
<i>B</i>	8	0	1	7	3	3
<i>C</i>	9	1	0	8	4	2
<i>D</i>	2	7	8	0	4	10
<i>E</i>	6	3	4	4	0	6
<i>F</i>	11	3	2	10	6	0

Compruébese que $D[A][F] = 11$, coherente con Dijkstra; además la matriz contiene las distancias entre todos los pares.

Limitaciones y supuestos (resumen)

- **Dijkstra:** requiere pesos no negativos; recomendado para un solo origen y muchos vértices (si se implementa con heap, es eficiente en grafos dispersos).
- **Floyd–Warshall:** $O(n^3)$ — útil para grafos con n moderado y cuando se desean distancias de todos los pares; soporta pesos negativos pero **no ciclos negativos**.
- Si existen pesos negativos y se necesita un único origen, usar **Bellman–Ford** (detecta ciclos negativos y tiene complejidad $O(nm)$).

Ejemplo contextualizado en Nicaragua

Escenario: Ciudad de reparto desde Managua a Matagalpa pasando por ciudades intermedias. Valores ilustrativos (no representan mediciones oficiales; son aproximaciones para el ejemplo).

Vértices

$V =$

{Managua (MGA), León (LEO), Granada (GRN), Masaya (MSY), Matagalpa (MTP), Jinotepe (JNP)}

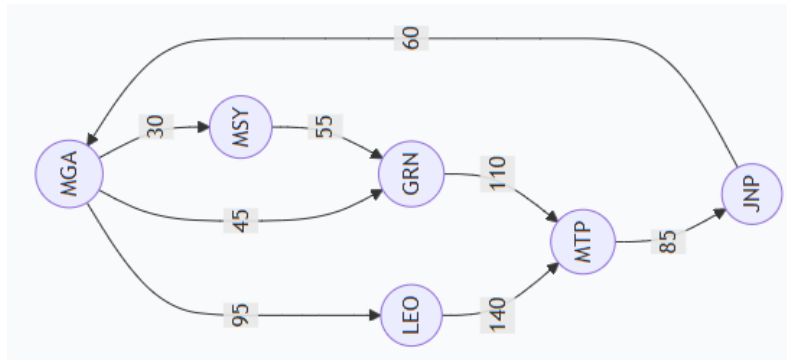
.

Aristas y distancias (km, hipotético):

- MGA–LEO = 95
- MGA–GRN = 45
- MGA–MSY = 30
- LEO–MTP = 140
- MSY–GRN = 55
- GRN–MTP = 110
- MTP–JNP = 85
- JNP–MGA = 60

graph TD

```
MGA((MGA)) --95--> LEO((LEO))
MGA((MGA)) --45--> GRN((GRN))
MGA((MGA)) --30--> MSY((MSY))
LEO((LEO)) --140--> MTP((MTP))
MSY((MSY)) --55--> GRN((GRN))
GRN((GRN)) --110--> MTP((MTP))
MTP((MTP)) --85--> JNP((JNP))
JNP((JNP)) --60--> MGA((MGA))
```



Pregunta: ¿Ruta más corta de Managua (MGA) a Matagalpa (MTP)?

Podemos aplicar Dijkstra (manualmente o con código). Resumen rápido (esquema de razonamiento):

- Desde MGA: $\text{dist}[\text{MGA}] = 0$; vecinos: LEO(95), GRN(45), MSY(30), JNP(60)
- Mejor candidato inicial es MSY (30), luego GRN (45), JNP (60), LEO (95).
- Relajando sucesivamente, se encontrará que el camino $\text{MGA} \rightarrow \text{GRN} \rightarrow \text{MTP}$ tiene coste $45 + 110 = 155$ km; $\text{MGA} \rightarrow \text{LEO} \rightarrow \text{MTP} = 95 + 140 = 235$ km; $\text{MGA} \rightarrow \text{JNP} \rightarrow \text{MTP} = 60 + 85 = 145$ km.
- Por tanto, **la ruta mínima (con los pesos dados) es $\text{MGA} \rightarrow \text{JNP} \rightarrow \text{MTP}$ con 145 km.**

Discusión: este ejemplo ilustra que la ruta con menos “saltos” (menos aristas) no siempre es la más corta; la conectividad y los pesos determinan la solución. Además, al diseñar rutas para logística, hay que considerar tiempos reales, tráfico, estado de carreteras y restricciones (p. ej. camiones con limitaciones).

Mejoras del ejemplo A→F: reconstrucción de ruta usando prev[] (mostrada arriba)

Vea la tabla de prev en la sección Dijkstra; la ruta se reconstruye retrocediendo desde destino hasta origen. Es una operación determinística en $O(\ell)$ donde ℓ es la longitud (en nodos) del camino.

Soluciones comentadas a los ejercicios propuestos

Ejercicio 1: Red de Transporte

Datos: Ciudades A,B,C,D con aristas: A-B:3, A-C:8, B-C:1, B-D:4, C-D:2.

a) Ruta más corta de A a D usando Dijkstra

Ejecución rápida:

- $\text{dist}[A]=0$; vecinos: B(3), C(8)
- Extraer B (3): relajar B-C: $\text{alt}=3+1=4 \rightarrow \text{dist}[C]=4$; relajar B-D: $\text{alt}=3+4=7 \rightarrow \text{dist}[D]=7$
- Extraer C (4): relajar C-D: $\text{alt}=4+2=6 \rightarrow \text{dist}[D]=6$ (mejora)
- Extraer D (6): terminado.

Resultado: $d(A,D)=6$; ruta $A \rightarrow B \rightarrow C \rightarrow D$.

b) Matriz de distancias mínimas usando Floyd-Warshall

Matriz final (orden A,B,C,D):

	A	B	C	D
A	0	3	4	6
B	3	0	1	3
C	4	1	0	2
D	6	3	2	0

(Se obtiene tras iteraciones; se puede verificar que $d(A,D)=6$).

Ejercicio 2: Sistema de Comunicación

Nodos: 1,2,3,4,5 con aristas y pesos indicados (hay aristas bidireccionales según listado).

a) **Camino más corto 1→4:** aplicar Dijkstra.

Resumen de cálculo (breve):

- $\text{dist}[1]=0$; vecinos: 2(10), 3(5)
- Extraer 3 (5): relajar $3 \rightarrow 2$ ($\text{alt}=8$) $\rightarrow \text{dist}[2]=8$ (mejora frente a 10). relajar $3 \rightarrow 5$ ($\text{alt}=7$) $\rightarrow \text{dist}[5]=7$.
- Extraer 2 (8): relajar $2 \rightarrow 4$ ($\text{alt}=9$) $\rightarrow \text{dist}[4]=9$.
- Extraer 5 (7): relajar $5 \rightarrow 4$ ($\text{alt}=13$) $\rightarrow$ no mejora ($9 < 13$). Extraer 4 (9): terminado.

Resultado: camino $1 \rightarrow 3 \rightarrow 2 \rightarrow 4$ con coste 9.

b) **Distancias mínimas todos los pares:** se recomienda correr Floyd–Warshall. Para la solución en clase proveer la matriz final y comprobar que $d(1,4)=9$.

Ejercicio 3: Problema de Logística (tiempos en minutos)

Oficinas: Central (C), Norte (N), Sur (S), Este (E), Oeste (O). Pesos indicados.

a) **Tiempo mínimo desde Central a todas las oficinas** — Dijkstra desde C.

Proceso (resumen):

- C vecinos: N(15), S(20), E(10), O(25) $\rightarrow$ dist inicial: N15,S20,E10,O25
- Extraer E(10): $E \rightarrow O$ $\text{alt}=10+6=16 \rightarrow$ mejora O:16 (antes 25)
- Extraer N(15): $N \rightarrow E$ $\text{alt}=15+8=23 \rightarrow$ no mejora E
- Extraer O(16): O no tiene aristas nuevas que mejoren
- Extraer S(20): $S \rightarrow E$ $\text{alt}=20+15=35$ no mejora

Resultado: $\text{dist}(C \rightarrow N)=15$, $C \rightarrow S=20$, $C \rightarrow E=10$, $C \rightarrow O=16$.

b) **Ruta más rápida entre cualquier par:** usar Floyd–Warshall para matriz completa; para pares concretos se puede usar Dijkstra repetido.

Ejercicio 4: Análisis comparativo (grafo dado)

Grafo: A–B:5, A–C:3, B–C:1, B–D:2, C–D:4, C–E:6, D–E:3.

a) **Distancia $A \rightarrow E$:** aplicar Dijkstra.

- $\text{dist}[A]=0$; vecinos: B5,C3
- Extraer C(3): $C \rightarrow D \text{ alt}=3+4=7 \rightarrow \text{dist}[D]=7$; $C \rightarrow E \text{ alt}=3+6=9 \rightarrow \text{dist}[E]=9$
- Extraer B(5): $B \rightarrow D \text{ alt}=5+2=7 \rightarrow \text{dist}[D]$ ya 7 $\rightarrow$ igual
- Extraer D(7): $D \rightarrow E \text{ alt}=7+3=10 \rightarrow$ no mejora ($9 < 10$)

Resultado: $d(A,E)=9$ via $A \rightarrow C \rightarrow E$.

b) **Verificación por Floyd–Warshall:** la matriz final debe tener $D[A][E]=9$; ambos algoritmos concuerdan.

Casos especiales y errores comunes

1. **Grafo no conexo:** Dijkstra desde un origen marcará $\text{dist}[v]=\infty$ para vértices no alcanzables. Floyd–Warshall dejará ∞ en las entradas correspondientes.
2. **Múltiples aristas entre dos nodos:** elegir la arista de menor peso (o transformarlas en una sola arista con peso mínimo).
3. **Ciclos de peso negativo:** Floyd–Warshall detecta ciclos negativos si alguna diagonal $D[i][i] < 0$ al final. Dijkstra no sirve en presencia de pesos negativos.
4. **Aristas con coste 0:** válidas; pueden crear caminos alternativos. No confundir coste 0 con ausencia de arista.
5. **Grafo dirigido vs no dirigido:** especificar dirección en entradas; convertir aristas no dirigidas a dos arcos si se trabaja con matrices dirigidas.
6. **Desbordamiento numérico:** en implementaciones reales, usar un valor grande para ∞ (por ejemplo $\text{INF} = 10^{18}$) y comprobar sumas que pudieran exceder tipos enteros.
7. **Errores de implementación comunes:** no usar `decrease_key` correcto en heap, olvidar inicializar prev, manejo inadecuado de $\infty + \text{peso}$.

Recomendación: para detectar errores de datos, visualizar el grafo y comprobar caminos candidatos manualmente para casos pequeños.

4.4. Algoritmo para flujo máximo en una red

4.4.1 Algoritmo de Ford y Fulkerson

4.5. Modelo de redes para proyectos

4.5.1 Técnica de Revisión y Evaluación de Proyectos (PERT)

4.5.2 Método de Ruta Crítica (CPM)

4.6. Aplicación de "Network Modelin" y "PERT_CPM" del WinOSB para la resolución de problemas de grafos y redes para proyectos contextualizados en la ingeniería de sistemas de información

Bibliografía y Webgrafía

Glosario